

Il tuo collega più produttivo scrive test inutili

Francesco Montelli · Freelance Software Engineer

DevRomagna

Tratto da una storia vera

Un progetto scritto a mano, anni fa.

Niente test perché tanto "ci pensiamo dopo".

Il tempo passa, il progetto rimane.

Poi qualcuno chiede: "lo usiamo ancora, ti va di aggiungerci cose?"

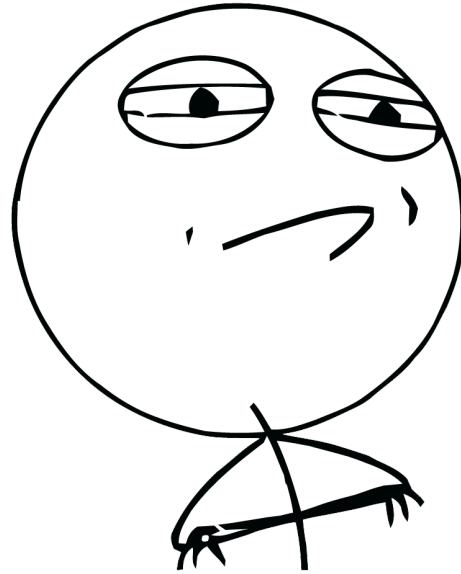
Francesco Montelli · Freelance Software Engineer

montelli.dev

Atto 1

I debiti si pagano

CHALLENGE ACCEPTED



Di che debito parliamo?

Debito tecnico

- Test mancanti o fragili
- Sistema che si rompe a ogni cambio
- Documentazione "gine"

Di che debito parliamo?

Debito tecnico

- Test mancanti o fragili
- Sistema che si rompe a ogni cambio
- Documentazione "gne"

Debito cognitivo / documentale

- I test sono **documentazione viva** del comportamento
- Soprattutto per **fallimenti** e **casi edge**
- Senza test → la conoscenza vive nella testa di qualcuno

Come ho ragionato

Punto di partenza

- Tempo poco, codebase grossa → serve scalabilità
- Dominio che conoscevo
- Memoria storica delle segnalazioni → so dove sono stati i problemi

Come ho ragionato

Punto di partenza: tempo poco, dominio noto, segnalazioni storiche come mappa

Cosa volevo evitare

- Test che **congelano il bug**
- Big-bang refactor
- **Scorciatoie:** mock ovunque, solo happy path
- Test troppo fattorizzati

I debiti si pagano... con gli interessi

- Wiki interna con specifiche e casi d'uso → **contesto** per gli agenti
Senza contesto, l'agente genera test generici

I debiti si pagano... con gli interessi

- Wiki interna con specifiche e casi d'uso → **contesto** per gli agenti
Senza contesto, l'agente genera test generici
- Test generati servizio per servizio, guidati dal contesto
Il ruolo cambia: pianificatore, architetto, controllore

I debiti si pagano... con gli interessi

- Wiki interna con specifiche e casi d'uso → **contesto** per gli agenti
Senza contesto, l'agente genera test generici
- Test generati servizio per servizio, guidati dal contesto
Il ruolo cambia: pianificatore, architetto, controllore
- CI pipeline: test + coverage report ad ogni push
Feedback continuo, non test lanciati a mano

Risultato

✓ Test tutti verdi

✓ Coverage alta

✓ Non li abbiamo scritti tutti noi



Atto 2

Wake up to reality

"Dopo l'ultimo aggiornamento si è rotto X"



Cosa dice davvero la coverage?

La coverage è una **metrica**: una rappresentazione della realtà.

Cosa dice davvero la coverage?

La coverage è una **metrica**: una rappresentazione della realtà.

Dice che siamo *passati* da quel punto.

Non dice che il test si accorgerà se qualcosa va male.

Eseguito $\neq$ verificato.

Quando è "corretto" un test?

Il test è verde. Ma perché?

1. È **vero**: testa ciò che promette di testare
2. È **affidabile**: passa *perché* l'implementazione è corretta

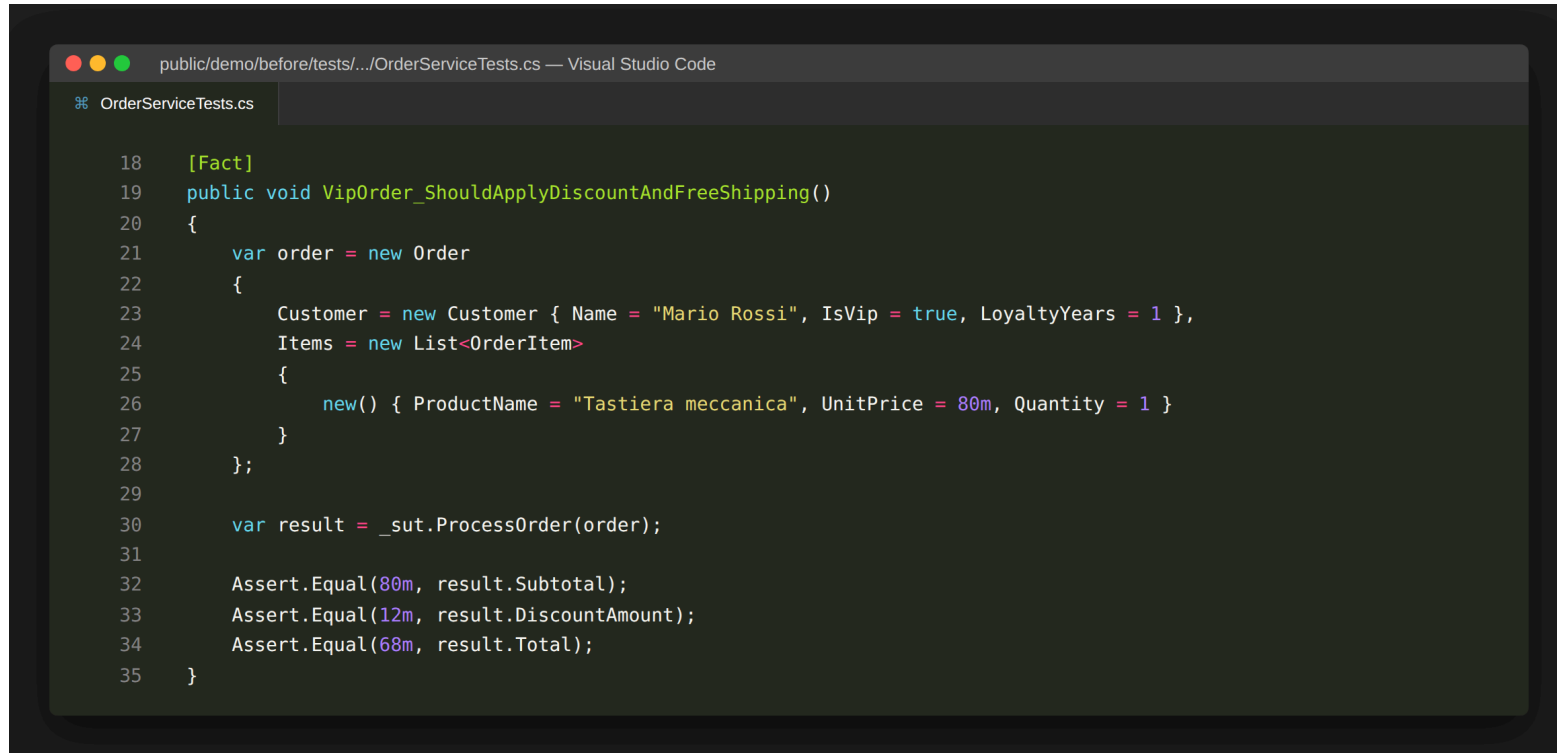
Concentriamoci sul secondo punto.

La chiave

Se il test passa *perché* l'implementazione è corretta...

...deve ****fallire**** se l'implementazione è sbagliata.

Verifichiamolo dal vivo



```
public/demo/before/tests/.../OrderServiceTests.cs — Visual Studio Code
OrderServiceTests.cs

18  [Fact]
19  public void VipOrder_ShouldApplyDiscountAndFreeShipping()
20  {
21      var order = new Order
22      {
23          Customer = new Customer { Name = "Mario Rossi", IsVip = true, LoyaltyYears = 1 },
24          Items = new List<OrderItem>
25          {
26              new() { ProductName = "Tastiera meccanica", UnitPrice = 80m, Quantity = 1 }
27          }
28      };
29
30      var result = _sut.ProcessOrder(order);
31
32      Assert.Equal(80m, result.Subtotal);
33      Assert.Equal(12m, result.DiscountAmount);
34      Assert.Equal(68m, result.Total);
35  }
```

public/demo/before/tests/MutationTestingDemo.Tests/OrderServiceTests.cs

public/demo/before/src/MutationTestingDemo/Services/OrderService.cs

Rompiamo il codice apposta

```
$ make test
docker compose -f public/demo/before/docker-compose.yml run --rm demo \
  -c "dotnet test --verbosity normal"

Restored /app/MutationTestingDemo.sln (in 0.4 sec).
  MutationTestingDemo -> /app/src/.../MutationTestingDemo.dll
  MutationTestingDemo.Tests -> /app/tests/.../MutationTestingDemo.Tests.dll

Test run for /app/tests/.../MutationTestingDemo.Tests.dll (.NETCoreApp,Version=v8.0)
Microsoft (R) Test Execution Command Line Tool Version 17.8.0
Copyright (c) Microsoft Corporation. All rights reserved.

Starting test execution, please wait...
A total of 1 test files matched the specified pattern.

Passed! - Failed:      0, Passed:      3, Skipped:      0, Total:      3, Duration: 28 ms

// Mutazione applicata: Sum() → Max() su OrderService.cs:26
// I test continuano a passare. Il mutante è "survived".
```

Rompiamo il codice apposta

Modifica 1: `Sum()` → `Max()`

- Il test ha **1 solo item** nel carrello
- `Sum(x) == Max(x)` con un solo elemento

Modifica 2: `* Quantity` → `/ Quantity`

- Il test usa **Quantity = 1**
- `price * 1 == price / 1`

Atto 3

Mutation testing

Romperlo a mano non scala

Utile sì (più confidenza), ma **tedioso ed error-prone**.

- Modificare il codice, lanciare i test, `git checkout` per ripristinare
- Una giornata a scrivere test è dura
- Una giornata a capire perché i test fanno schifo è **peggio**

Romperlo a mano non scala

Utile sì (più confidenza), ma **tedioso ed error-prone**.

- Modificare il codice, lanciare i test, `git checkout` per ripristinare
- Una giornata a scrivere test è dura
- Una giornata a capire perché i test fanno schifo è **peggio**

Esiste già. Si chiama mutation testing.

- Fa esattamente quello che abbiamo fatto a mano
- Una libreria per ogni linguaggio · funzionalità simili

Stryker.NET

Quello che abbiamo fatto a mano su 2 righe,
Stryker lo fa su **tutta la codebase**.

- Analizza il sorgente, genera le varianti, lancia i test
- Open source · .NET · `dotnet tool install -g dotnet-stryker`

Stryker.NET

Quello che abbiamo fatto a mano su 2 righe,
Stryker lo fa su **tutta la codebase**.

- Analizza il sorgente, genera le varianti, lancia i test
- Open source · .NET · `dotnet tool install -g dotnet-stryker`

Cosa offrono i tool (Stryker incluso)

- Tipi di mutazione · cosa escludere · **report navigabili**
- Il report ti porta sul punto fragile, riga per riga

Come funziona il mutation testing

Step	Azione
1	Analizza il sorgente → identifica i punti mutabili
2	Genera i mutanti → una copia del codice per ogni modifica
3	Lancia i test contro ogni mutante
4	Test fallisce = killed · Test passa = survived

Survived = buco nei test

Come si legge il report

Tre livelli di navigazione

- **Dashboard** : score complessivo + per file (dove sto messo peggio?)
- **File** : sorgente con pallini colorati a fianco di ogni riga
- **Mutante** : diff della modifica + esito dei test

Stati dei mutanti

Stato	Significato
Killed	Test rosso : la rete ha funzionato
Survived	Test verdi : buco da indagare
No coverage	Riga mai eseguita dai test
Timeout / Compile error	Casi limite, conteggiati come killed

Mutation score

All files Stryker.NET Report

Mutants

Tests

All files

26

12

2

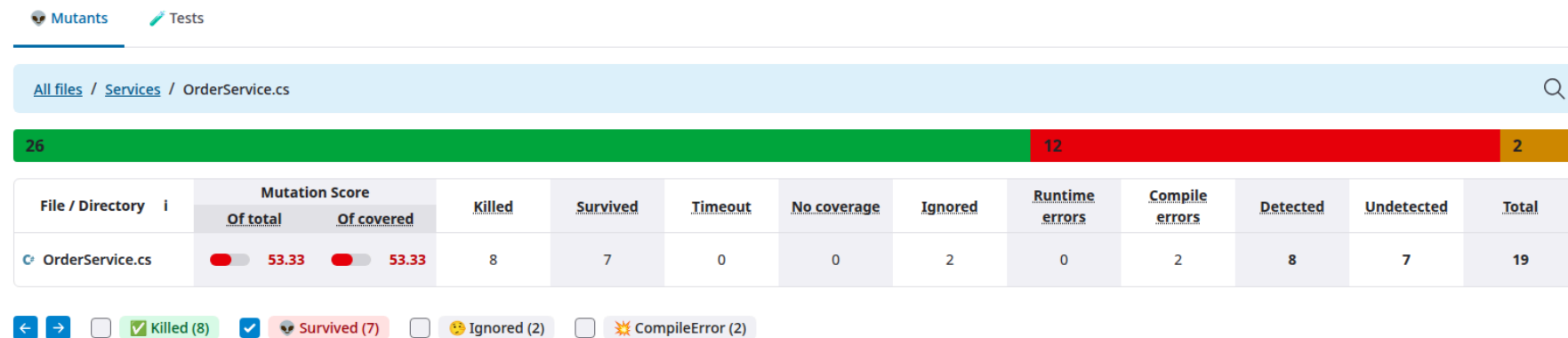
File / Directory	i	Mutation Score		Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Undetected	Total
		Of total	Of covered										
All files		65.00	68.42	26	12	0	2	7	0	2	26	14	49
Exceptions/InvalidOrderException.cs		N/A	N/A	0	0	0	0	0	0	0	0	0	0
Interfaces		N/A	N/A	0	0	0	0	0	0	0	0	0	0
Models		N/A	N/A	0	0	0	0	0	0	0	0	0	0
Services		65.00	68.42	26	12	0	2	7	0	2	26	14	49

Il **35%** dei mutanti è sopravvissuto.



I survived

OrderService.cs Stryker.NET Report



```
1 using MutationTestingDemo.Exceptions;
2 using MutationTestingDemo.Interfaces;
3 using MutationTestingDemo.Models;
4
5 namespace MutationTestingDemo.Services;
6
7 public class OrderService
8 {
9     private readonly IDiscountService _discountService;
10    private readonly IShippingService _shippingService;
11
12    public OrderService(IDiscountService discountService, IShippingService shippingService)
13    {
14        _discountService = discountService;
15        _shippingService = shippingService;
16    }
17
18    public OrderResult ProcessOrder(Order order)
19    {
20        if (order.Items == null || order.Items.Count == 0)
21        {
```


I survived

DiscountService.cs Stryker.NET Report

 Mutants  Tests

[All files](#) / [Services](#) / DiscountService.cs 

26

12

2

File / Directory	Mutation Score		Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Undetected	Total
	Of total	Of covered										
 DiscountService.cs	73.33	78.57	11	3	0	1	4	0	0	11	4	19

  ☐  Killed (11) ☒  Survived (3) ☒  NoCoverage (1) ☐  Ignored (4)

```
1 using MutationTestingDemo.Interfaces;
2 using MutationTestingDemo.Models;
3
4 namespace MutationTestingDemo.Services;
5
6 public class DiscountService : IDiscountService
7 {
8     public decimal CalculateDiscount(Customer customer, decimal subtotal)
9     {
10         decimal discountPercentage = 0m;
11
12         // VIP: +15% sconto
13         if (customer.IsVip)
14         {
15             discountPercentage += 0.15m;
16         }
17
18         // Ordine > 200 EUR: +5% sconto
19         if (subtotal > 200m) ●
20         {
21             discountPercentage += 0.05m;
```

I survived

1. Dato troppo semplice

`Sum()` → `Max()` : con 1 solo item, sono equivalenti

2. Assertion incompleta

Status mutato in tutti i modi, nessun test legge `result.Status`

3. Boundary mancante

`>= 3 anni` → `> 3 anni` : test usa `LoyaltyYears = 5` , mai 3

⚠ Non tutti i survived sono bug: alcuni sono **mutanti equivalenti**.
Per questo il 100% non è l'obiettivo.

Fix

Non servono test a caso. Servono *test guidati dal report*.

OrderServiceTests.cs

```
11  [Fact]
12  public void VipOrder_ShouldCalculateCorrectTotals()
13  {
14      var order = new Order
15      {
16          Customer = new Customer { Name = "Mario Rossi", IsVip = true, LoyaltyYears = 1 },
17          Items = new List<OrderItem>
18          {
19              new() { ProductName = "Tastiera meccanica", UnitPrice = 80m, Quantity = 2 },
20              new() { ProductName = "Mouse wireless",      UnitPrice = 40m, Quantity = 1 }
21          }
22      };
23
24      var result = _sut.ProcessOrder(order);
25
26      // Subtotale: 80*2 + 40*1 = 200 (multi-item killa Sum->Max)
27      Assert.Equal(200m, result.Subtotal);
28      // VIP 15% di 200 = 30
29      Assert.Equal(30m, result.DiscountAmount);
30      // VIP -> spedizione gratuita
31      Assert.Equal(0m, result.ShippingCost);
32      Assert.Equal(ShippingType.Free, result.ShippingType);
33      // Totale: 200 - 30 + 0 = 170
34      Assert.Equal(170m, result.Total);
35      // Status confermato
36      Assert.Equal(OrderStatus.Confirmed, result.Status);
37  }
```

Risultati

All files Stryker.NET Report

Mutants

Tests

All files

373

File / Directory	Mutation Score		Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Undetected	Total
	Of total	Of covered										
All files	92.50	92.50	37	3	0	0	7	0	2	37	3	49
Exceptions/InvalidOrderException.cs	N/A	N/A	0	0	0	0	0	0	0	0	0	0
Interfaces	N/A	N/A	0	0	0	0	0	0	0	0	0	0
Models	N/A	N/A	0	0	0	0	0	0	0	0	0	0
Services	92.50	92.50	37	3	0	0	7	0	2	37	3	49

Risultati

BEFORE

65%

12 survived

AFTER

92%

3 survived



Il report è uno specchio

I survived sono **diagnosi sui test**, non solo bug nel codice.

Categoria	Cosa rivela
Dati uniform / boundary mancanti	input non discriminanti
Assertion incompleta	output verificato a metà
Over - mocking	testi i mock, non il sistema
Overspecification	assert legati alla forma, non all'intento

Quattro pattern, quattro esempi

1. Dati uniform / boundary mancanti

```
var order = new Order {  
    Items = [ new() { Price = 80m, Qty = 1 } ]  
};  
// Sum() → Max()  x  con 1 elemento sono uguali
```

Quattro pattern, quattro esempi

2. Assertion incompleta

```
var r = svc.Process(order);  
Assert.Equal(170m, r.Total);  
// nessun assert legge r.Status  x
```

Quattro pattern, quattro esempi

3. Over-mocking

```
mockDiscount
    .Setup(x => x.Compute(It.IsAny<Order>()))
    .Returns(30m);
// le mutazioni in DiscountService → no coverage x
```

Quattro pattern, quattro esempi

4. Overspecification

```
Assert.Equal("Total: 100 EUR", r.ToString());  
// refactor di forma → test rosso, comportamento OK x
```

Blindspot 1: Assegnazione sbagliata

```
return new OrderResult {  
    Subtotal = order.Subtotal,  
    Total     = order.Subtotal,    // ← bug: doveva essere `computed`  
};
```

- Sintatticamente **identica** al codice corretto
- Stryker non muta un identificatore in un altro identificatore diverso
- Tipicamente nasce da **copia-incolla** o autocomplete
- I test possono passare se Subtotal e Total coincidono per i casi testati

Lo trova **solo un occhio umano** sul diff, o un test che differenzia esplicitamente i due campi.

Blindspot 2: Method chain / Fluent API

```
var ids = invoices.Where(i => i.IsOverdue)
                  .Select(i => i.Id)
                  .First();           // ← dovrebbe essere ToList()
```

- Stryker muta operatori e costanti, **non** intercambia metodi LINQ
- `.First()` vs `.ToList()` hanno **firme diverse** ma logica completamente diversa
- Errori tipici: `First` invece di `Single`, `Any` invece di `All`, `OrderBy` invece di `OrderByDescending`
- Il test può passare se l'input ha **un solo elemento qualificante**

Si presidiano con **test sul comportamento di insieme** (più di un elemento, ordering significativo) e con code review attento alle catene LINQ.

Oltre il codice: cosa nessuna mutazione vede

Stryker muta **sintassi**. Alcune cose vivono altrove.

- **Concorrenza** : race condition, ordering, deadlock
- **Performance** : $O(n^2)$ vs $O(n \log n)$, stesso output
- **Configurazione** : env, feature flag, secret
- **Comportamento di sistema** : timeout di rete, lock DB, auth reale

Mutation testing è un livello. Sotto: unità. Sopra: **E2E, integrazione, observability**.

Triage del report

Davanti a un report, in 3 passaggi:

1. **Priorità per file** : business / soldi / dati utente prima
2. **Per ogni survived** :
 - mutazione *plausibile* in produzione?
 - quale **categoria**? (dati · assert · mock · overspec)
 - test mirato da scrivere, o equivalente?
3. **Ignora consapevolmente** : generato, equivalent, no-coverage di codice noto morto

Non inseguire il 100%. **70-85% pulito** vale di più di 95% rumoroso.

Atto 4

L'automazione

Non è un pasto gratis

Per ogni mutante: genera una variante, esegui i test.

$$N \text{ mutanti} \times T_{\text{suite}} = \text{tempo totale}$$

Sulla demo: ~30 sec. Su una codebase reale: ore.

Ma c'è un'altra lettura:

Il mutation testing non **crea** lentezza: la **rivela**.

Suite lente, test sovrapposti, test non parallelizzabili: esistevano già.

Mutare solo dove conta

stryker-config.json

```
{
  "stryker-config": {
    "mutate": [
      "src/**/*.Service.cs",
      "src/**/*.Handler.cs",
      "!src/**/*.Migrations/**"
    ],
    "ignore-methods": [
      "*Log*", "Console.Write*", "nameof"
    ]
  }
}
```

Concentra le mutazioni sulla **logica di business**.

Quality gate in CI

L'ottimizzazione più importante: muta solo il codice cambiato

```
dotnet stryker --since:main
```

Soglia pragmatica:

```
"thresholds": { "high": 80, "low": 60, "break": 60 }
```

GitHub Actions:

```
- name: Mutation Testing  
  run: dotnet stryker --since:main
```

Ogni PR verificata automaticamente.

Atto 5

L'evoluzione delle automazioni

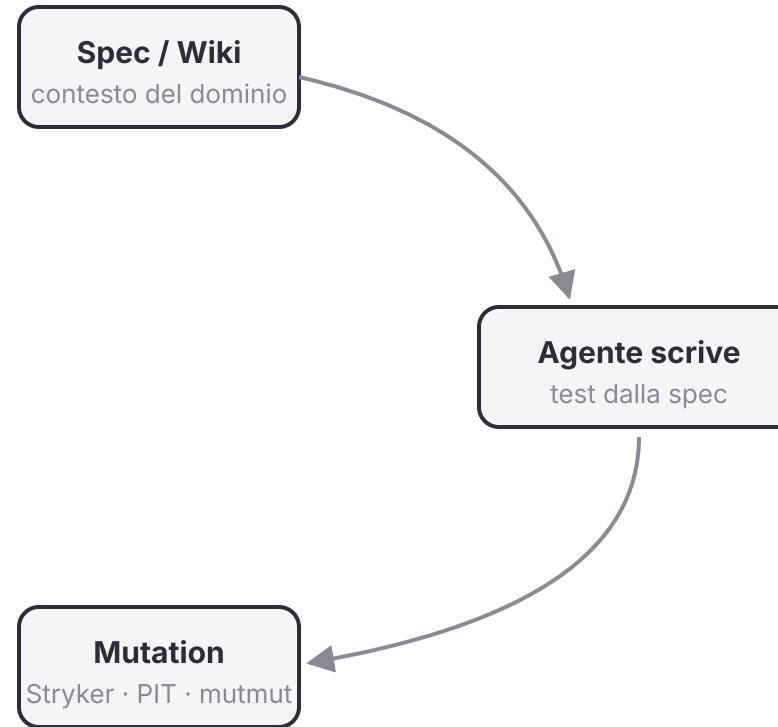
Più codice. Meno certezze.

Negli ultimi anni gli LLM hanno **moltiplicato** la produzione di codice.

E con essa, la quantità di test che **sembrano coprire** ma non verificano.

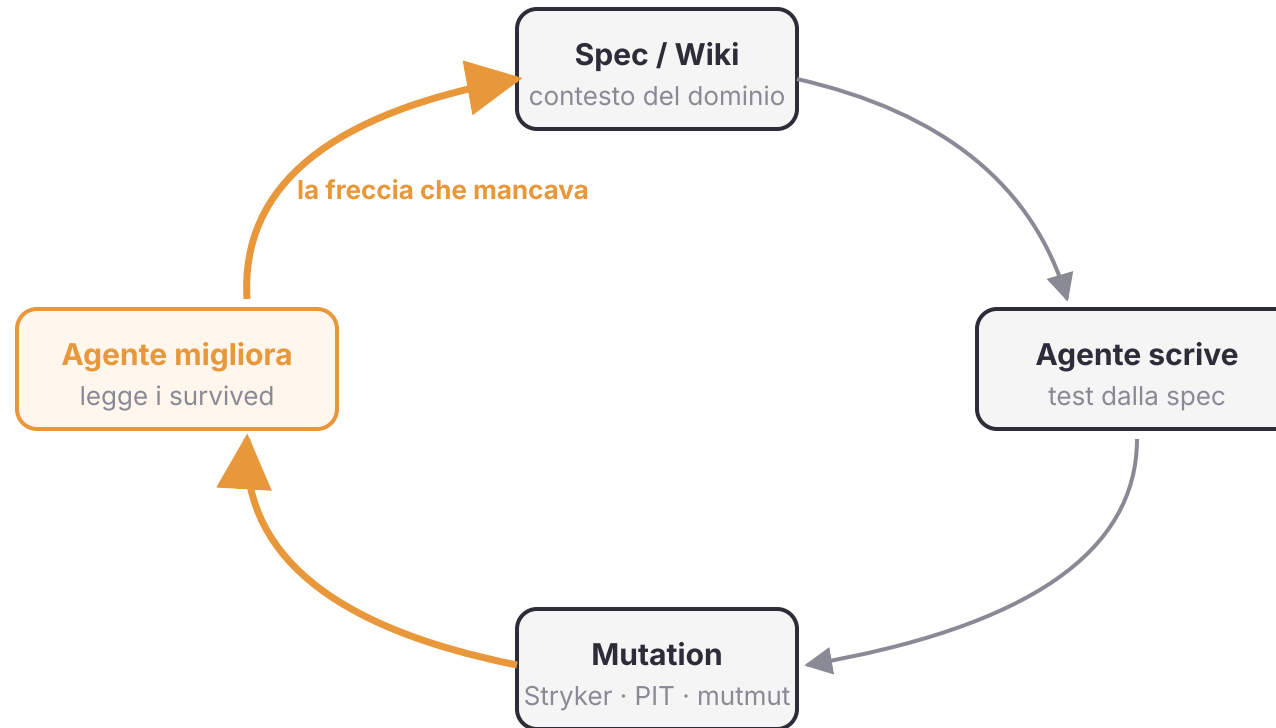


Il loop (aperto)



Stryker ci dà il report. Ma **chi lo legge?**

Il loop (chiuso)



Mutanti che hanno un *senso*

STRYKER CLASSICO

Mutanti generici

`+` $\rightarrow$ `-`

`>` $\rightarrow$ `>=`

`true` $\rightarrow$ `false`

`Sum` $\rightarrow$ `Max`

`a && b` $\rightarrow$ `a || b`

...tutto a tappeto

LLM-GUIDED (META ACH)

Mutanti *per area di test*

Privacy $\rightarrow$

`log(user.email)`

`skip(consent.check)`

Security $\rightarrow$

`decrypt && return raw`

`role.bypass()`

Lo stesso LLM riconosce gli equivalenti

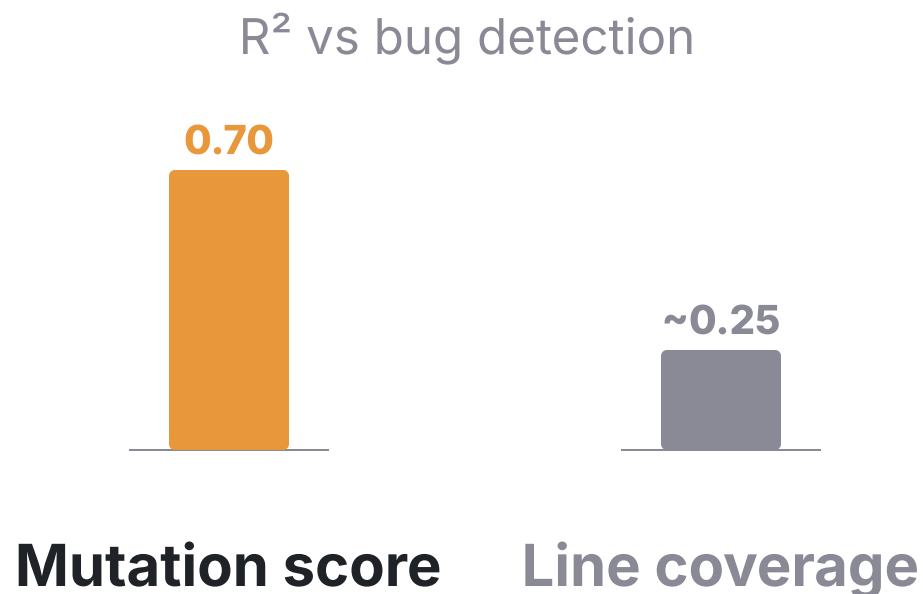


Per **30 anni** il muro del 100%. Adesso un modello che ha imparato la **firma semantica** lo separa.

Perché vale la pena

Una test suite che **uccide più mutanti** trova davvero più bug reali?

Just et al., FSE 2014: correlazione su 5 progetti open source, bug storici reali.



*Il mutation score è il **predittore più forte** di bug detection nello studio: quasi **3× rispetto alla line coverage**, su 357 bug reali estratti dalla storia git dei progetti.*

Conclusioni

Torniamo all'inizio

Coverage al 93%. Test tutti verdi.

Bug in produzione per tre settimane.

Mutation score: 65%. Tra i survived: `Sum→Max`.

Dodici test riscritti. Score: 92%.

Non è una storia sull'aggiungere test.
È una storia sull'**ascoltare i segnali giusti**.

La tesi

La **coverage** vi dice se il codice viene **eseguito**.

Il **mutation score** vi dice se i test **funzionano**.

Sono due domande diverse. Vale la pena farle entrambe.

Cosa fare domani

Scegli un servizio: quello con la logica più critica.

Lanciatelo. Leggete i survived come domande che i vostri test non si erano ancora posti.

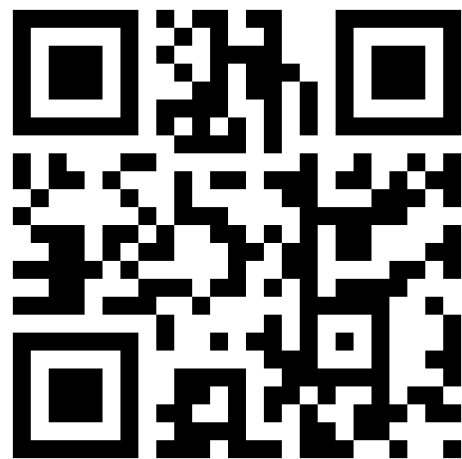
```
dotnet tool install -g dotnet-stryker  
dotnet stryker
```

Anche senza il tool, la **mentalità** ti cambia il code review.



It ain't much, but it's honest work

Francesco Montelli · Freelance Software Engineer



CONTATTI



MATERIALE

Domande?
